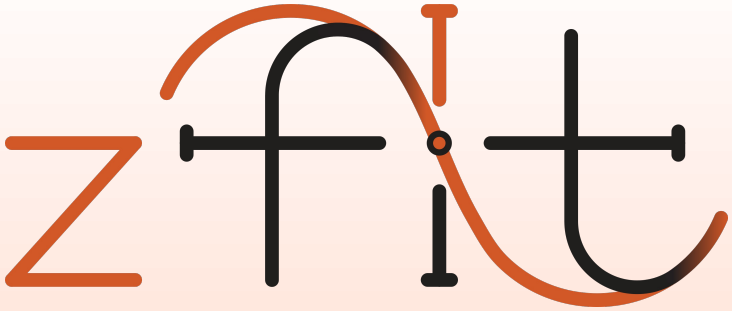
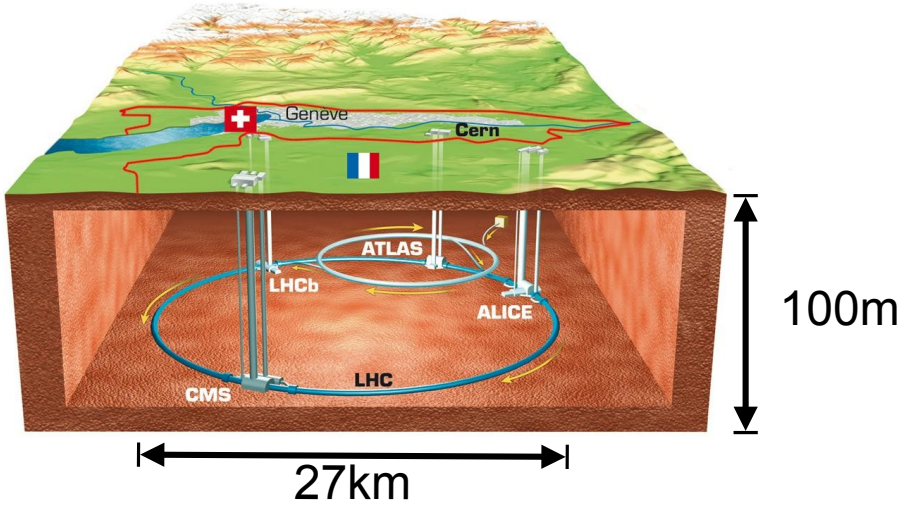
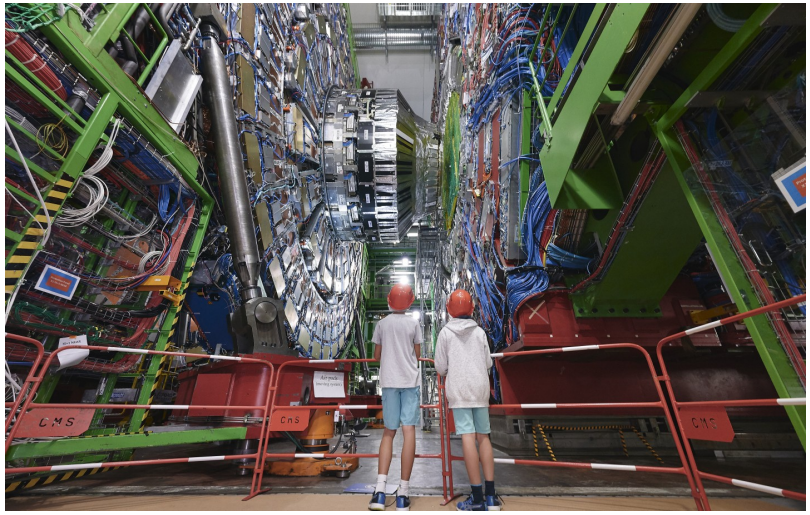


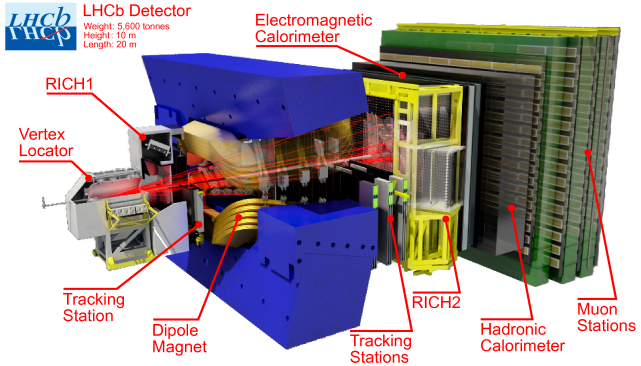
scalable pythonic likelihood fitting

Jonas Eschle
jonas.eschle@cern.ch
for the zfit team

High Energy Physics

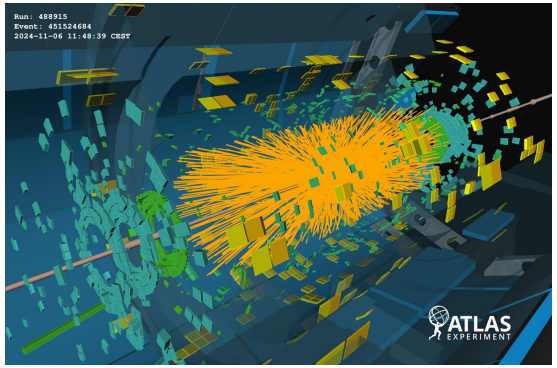




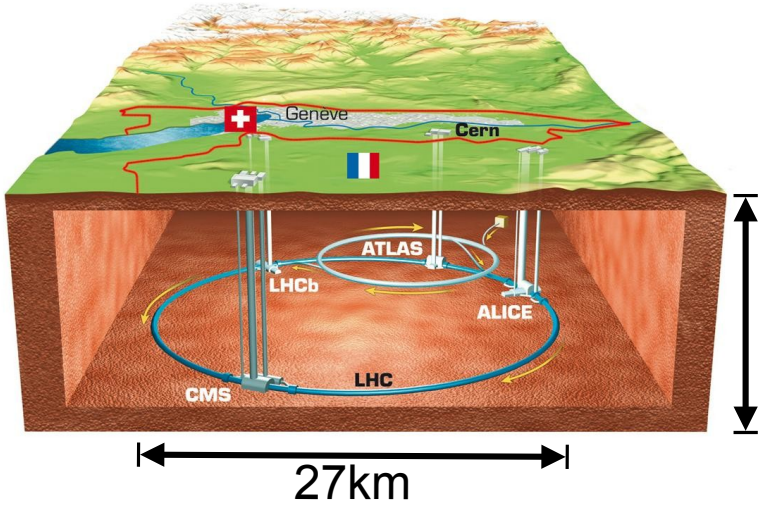


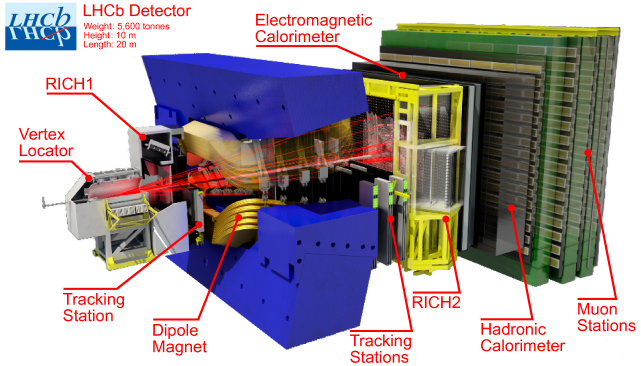
15m

Every 25 nano sec

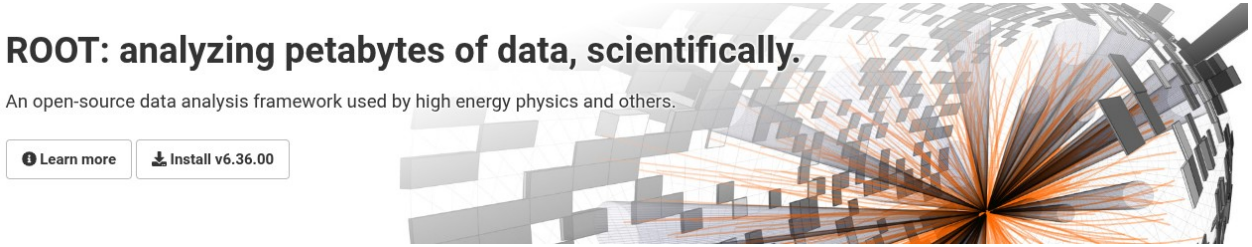


High Energy Physics

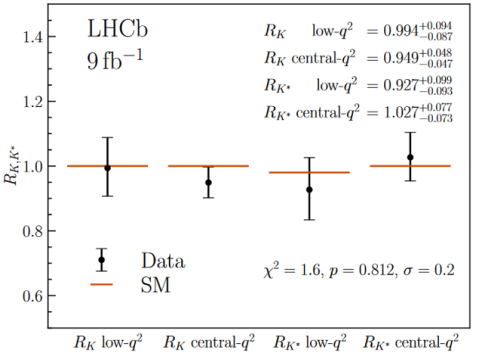


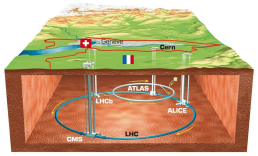


HEP Analysis

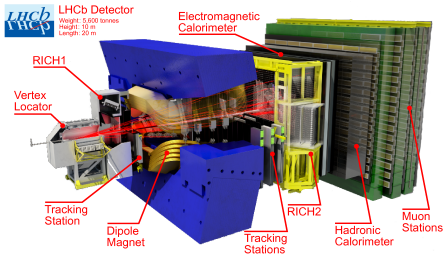


C++ large data analysis framework (1994)

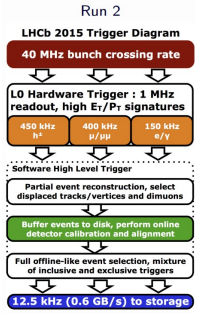




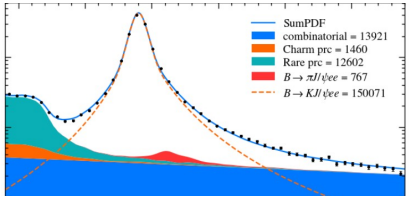
Lots of code



Terabytes per second



Petabytes of data



Lots of code

Lots of code

Historic landscape

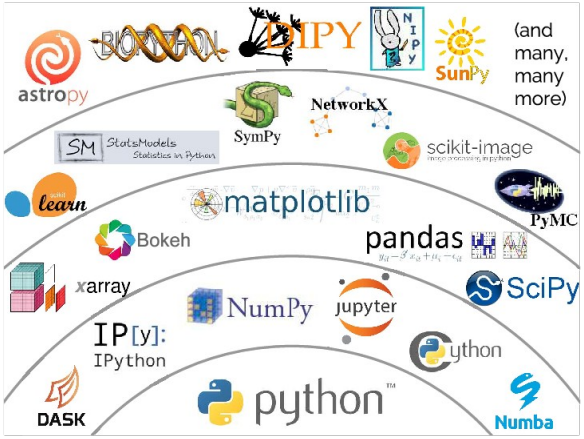
Analyses transition from C++ to Python

HEP (fitting) libraries still in C++
Good libraries
mediocre bindings
not «pythonic»

Leveraging Python

Analyses transition from C++ to Python

Provides a lot of functionality
pandas, scipy & friends



Missing:
- *domain specific packages*
  particle data, plotting styles,…
- *fundamentals for big data*
  histograms, fitting, jagged arrays

2016

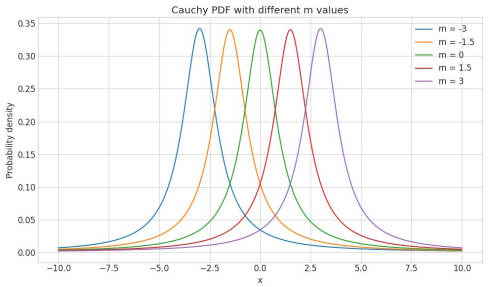
Likelihood fitting

in High Energy Physics

Likelihood-like fitting

$\mathrm{p}(\Omega) = 1$: *The probability over the whole sample space* Ω *is* 1

PDF are normalized



convert arbitrary function f(x)

$$PDF_{\mathrm{f}}(\mathrm{x}) = \frac{\mathrm{f}(\mathrm{x})}{\int_{\Omega} \mathrm{f}(\mathrm{x})\mathrm{dx}}$$

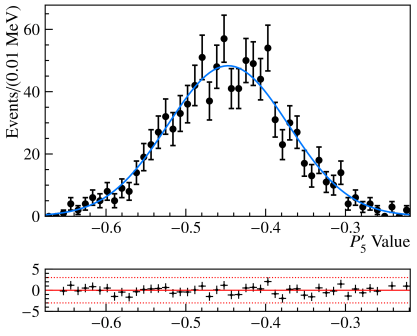
$$\mathcal{L}(\theta) = p_1(x_1;\theta_1) \cdot p_2(x_2;\theta_2) \cdot p_3(x_3;\theta_3)... = \prod_i^n p_i(x_i;\theta_i)$$

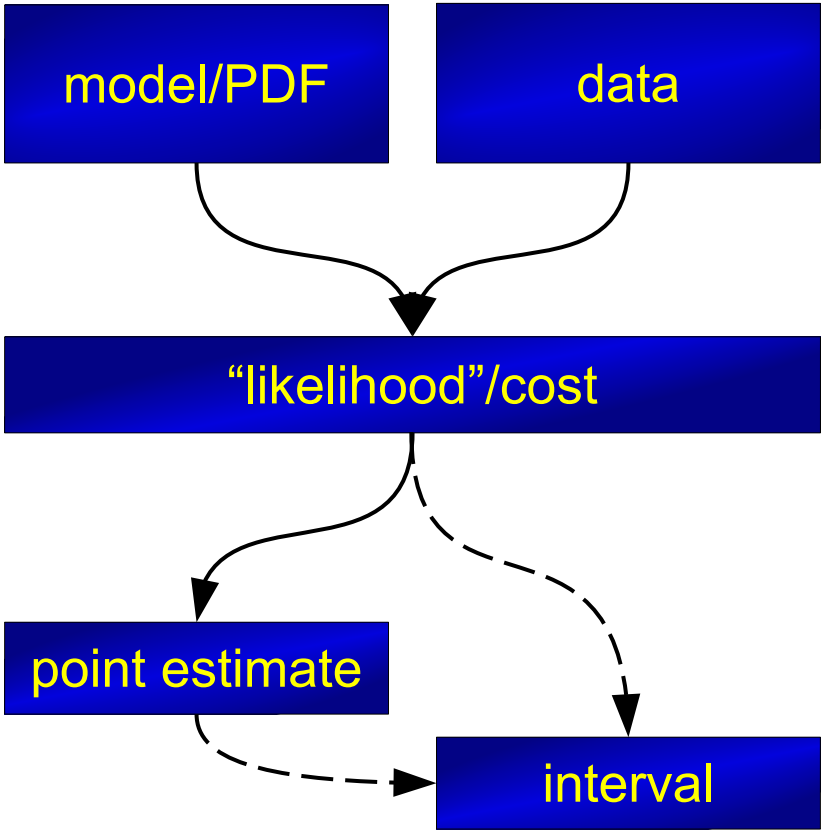
Maximize likelihood → «fitting»

or minimize negative log likelihood

$$\mathrm{NLL}(\theta) \equiv -\log \mathcal{L}(\theta) = -\sum_{\mathrm{i}=1}^{\mathrm{n}} \log \mathrm{p}(\mathrm{x_i}|\theta)$$

Parameter uncertainty estimation





Different kind of fits

- **Data:** Binned vs unbinned

Different kind of fits

- Data: Binned vs unbinned

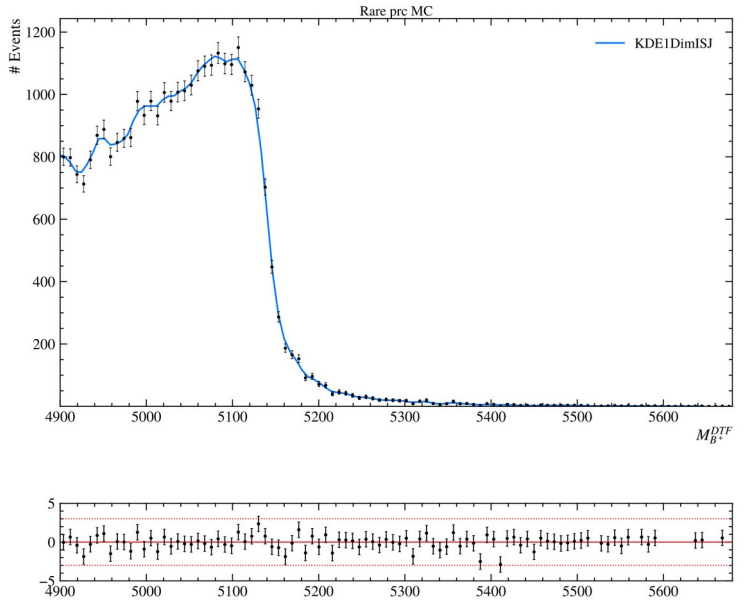
- **Shape:** Template vs analytic

Different kind of fits

- Data: Binned vs unbinned
- Shape: Template vs analytic
- ***Normalization:*** *Analytical vs numerical*

Different kind of fits

- Binned vs unbinned
- Template vs analytic
- *Analytical* vs *numerical*



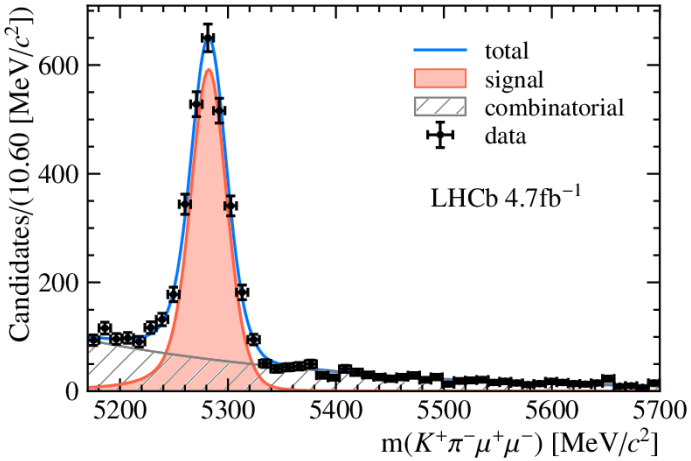
KDE

Gaussian kernel → analytic norm

ISJ → numeric norm

Different kind of fits

- Binned vs unbinned
- Template vs analytic
- *Analytical vs numerical*



Gauss + Exponential

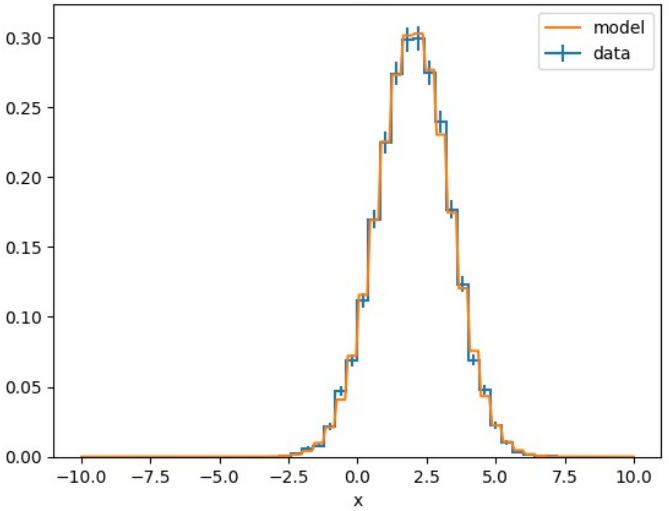
Composite model

Normalization range!

$$PDF_{\mathrm{f}}(\mathrm{x}) = \frac{\mathrm{f}(\mathrm{x})}{\int_{\Omega} \mathrm{f}(\mathrm{x})\mathrm{dx}}$$

Different kind of fits

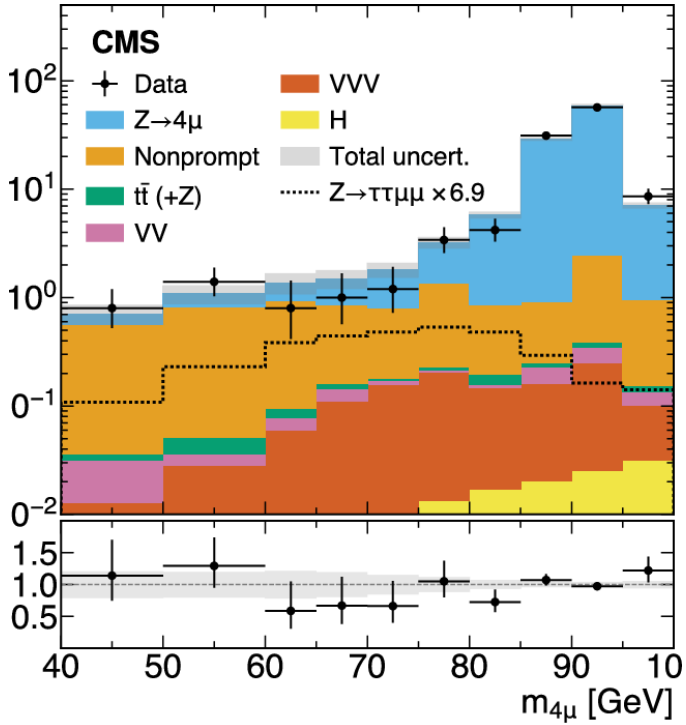
- Binned vs unbinned
- Template vs analytic
- *Analytical vs numerical*



(binned) Gaussian fit to histogram

Different kind of fits

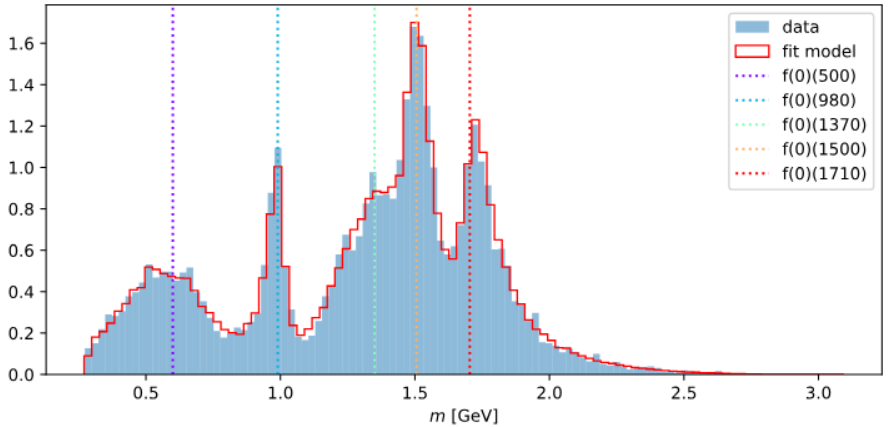
- Binned vs unbinned
- Template vs analytic
- *Analytical vs numerical*



Stacked histograms PDFs

Different kind of fits

- Binned vs unbinned
- Template vs analytic
- Analytical vs numerical



Amplitude analysis,
Angular analysis

Multidimensional custom models

Numerical methods

Custom models, compositions don’t have analytic integral

Numerical methods first class support

- Required for integration (normalization!), sampling

$$PDF_{\mathrm{f}}(\mathrm{x}) = \frac{\mathrm{f}(\mathrm{x})}{\int_{\Omega} \mathrm{f}(\mathrm{x})\mathrm{dx}}$$

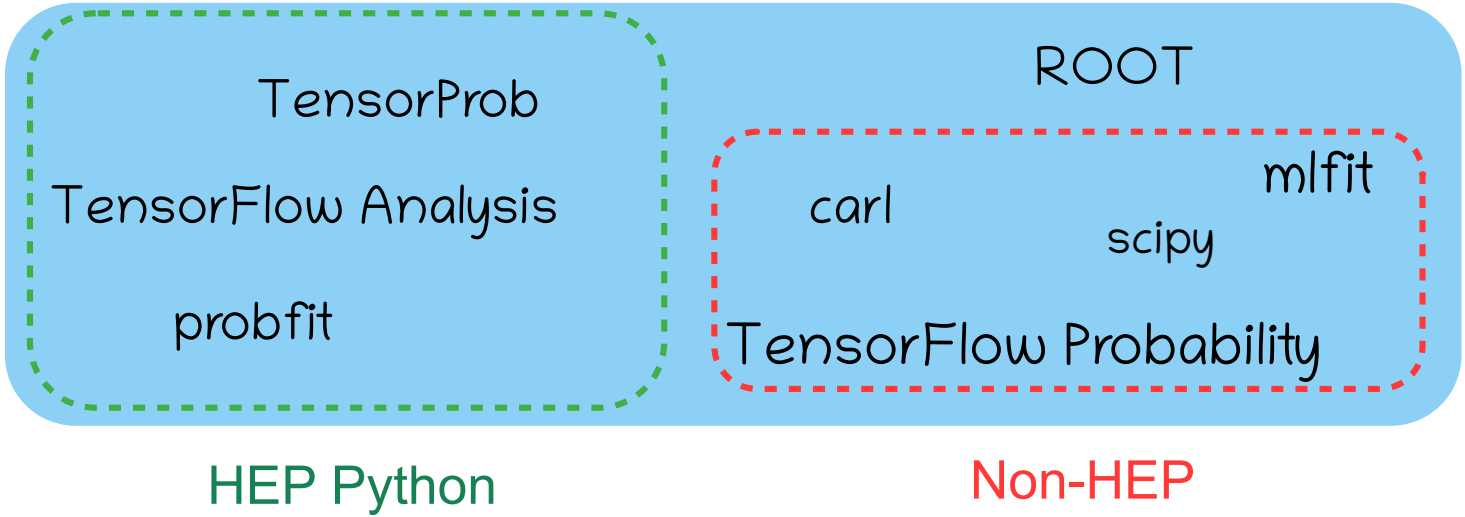
- Needs specific distribution API:
  - integrate, cannot cdf(upper) – cdf(lower)
  - sample, no icdf
  - also «needs» specific range (“cannot” integrate from -inf to inf)

Historic landscape

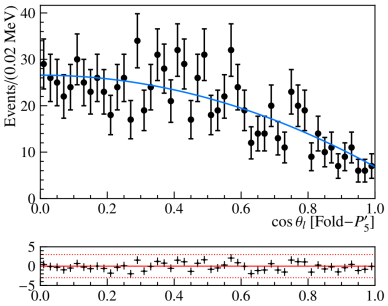
Analyses transition from C++ to Python

HEP fitting libraries still in C++
Good libraries
mediocre bindings
not «pythonic»

Python packages
- no advanced features
- performance needed

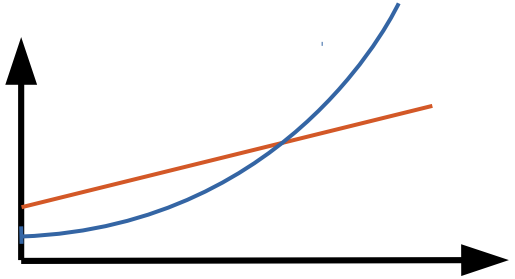


HEP Model Fitting in Python



HEP

advanced features,
simply extendable



Scalable

large data, complex models

Scikit HEP

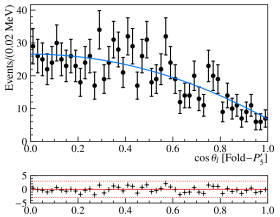
Pythonic

NumPy

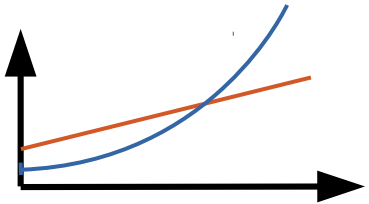
python™

integrate into ecosystem, stable API

HEP Model Fitting in Python



HEP
advanced features,
simply extendable

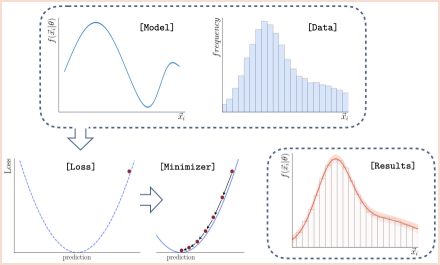


Scalable
large data, complex models

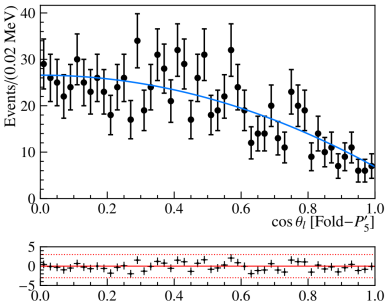
Pythonic
integrate into ecosystem, stable API

Computing backend

API & Workflow

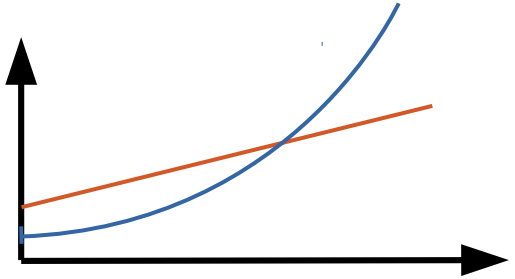


HEP Model Fitting in Python



HEP

advanced features,
simply extendable



Scalable

large data, complex models

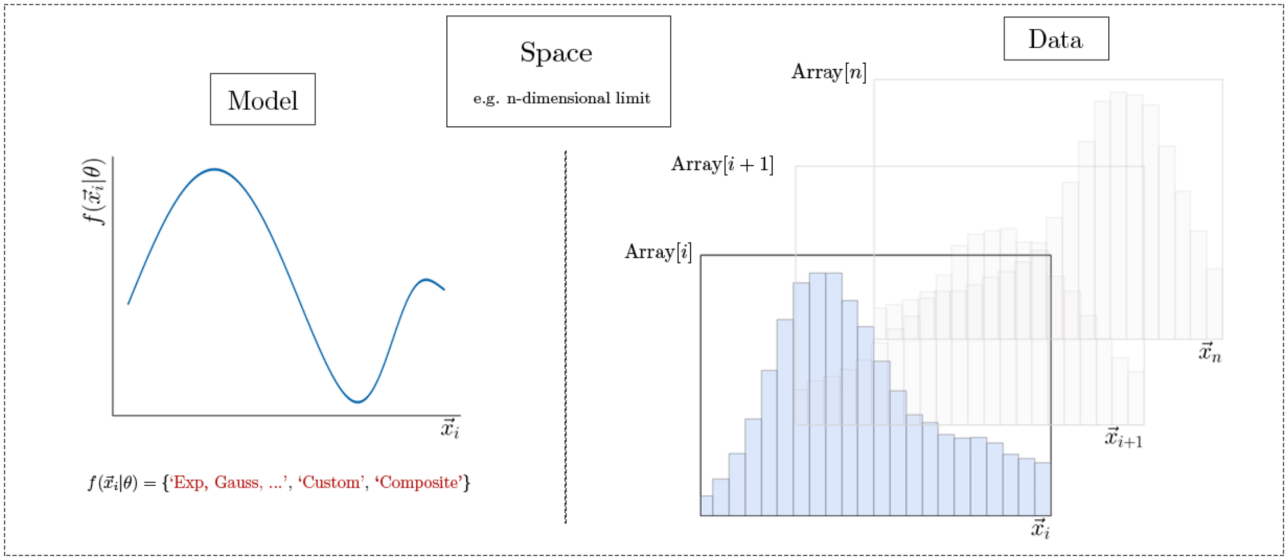
Pythonic

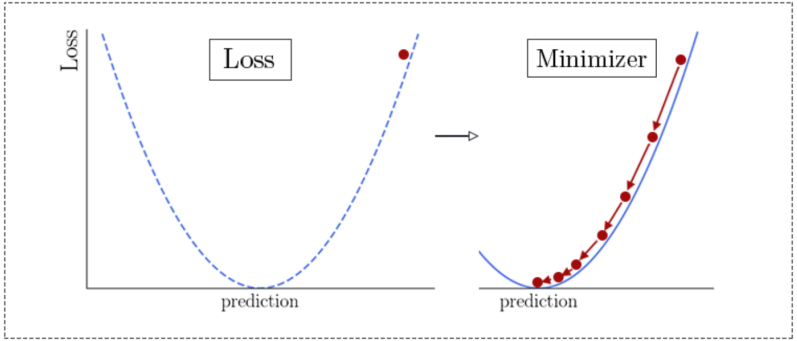
integrate into ecosystem, stable API

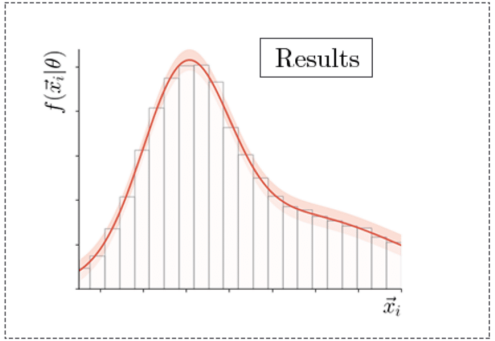
API & Workflow

Five maximally independent parts

"Fits look always the same"

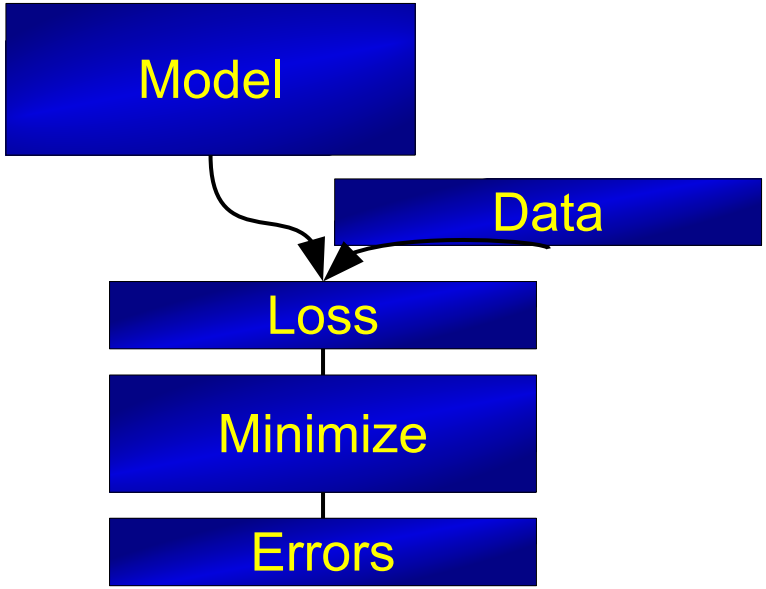






Basic API example

```
obs = zfit.Space("x", -2, 3)

mu = zfit.Parameter("mu", 1.2, -4, 6)
sigma = zfit.Parameter("sigma", 1.3, 0.5, 10)
gauss = zfit.pdf.Gauss(mu=mu, sigma=sigma, obs=obs)

data = zfit.Data(data, obs=obs)

nll = zfit.loss.UnbinnedNLL(model=gauss, data=data)

minimizer = zfit.minimize.Minuit()
result = minimizer.minimize(nll)

param_errors = result.hesse()
param_errors_asymmetric, new_result = result.errors()
```



Basic API example

Going binned

```
obs = zfit.Space("x", -2, 3, binning=30)
```

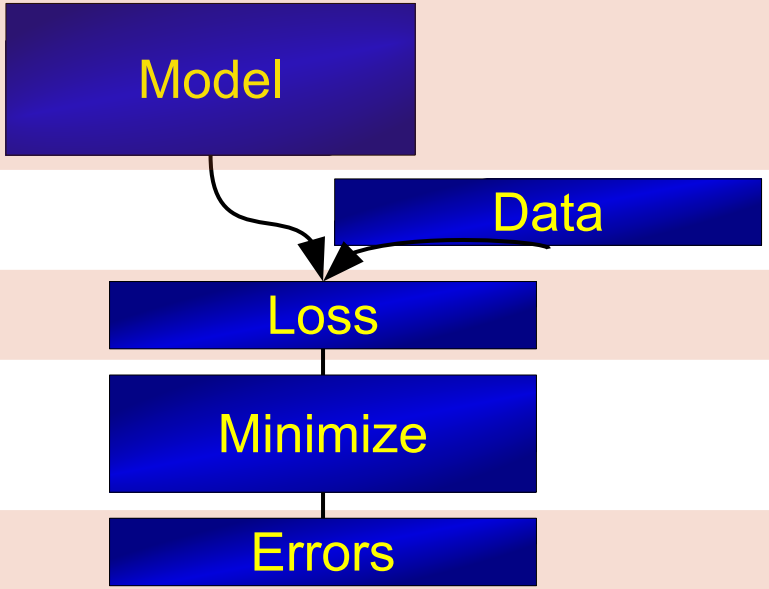
```
mu = zfit.Parameter("mu", 1.2, -4, 6)
sigma = zfit.Parameter("sigma", 1.3, 0.5, 10)
gauss = zfit.pdf.Gauss(mu=mu, sigma=sigma, obs=obs)
```

```
data = zfit.Data(data, obs=obs)
```

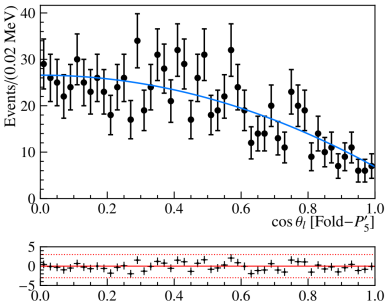
```
nll = zfit.loss.BinnedNLL(model=gauss, data=data)
```

```
minimizer = zfit.minimize.Minuit()
result = minimizer.minimize(nll)
```

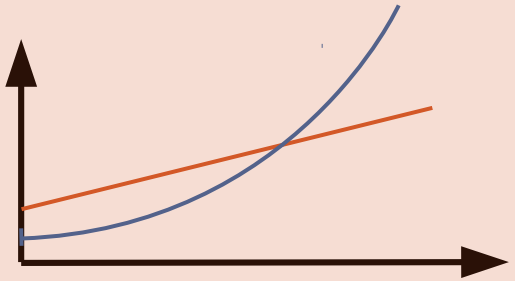
```
param_errors = result.hesse()
param_errors_asymmetric, new_result = result.errors()
```



HEP Model Fitting in Python



HEP
advanced features,
simply extendable



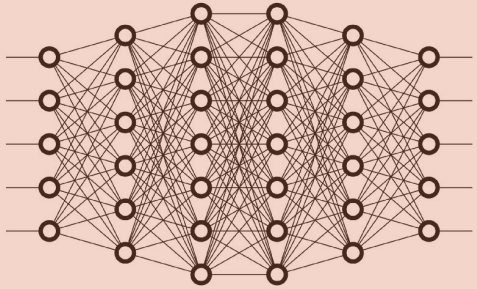
Scalable
large data, complex models

Scikit HEP

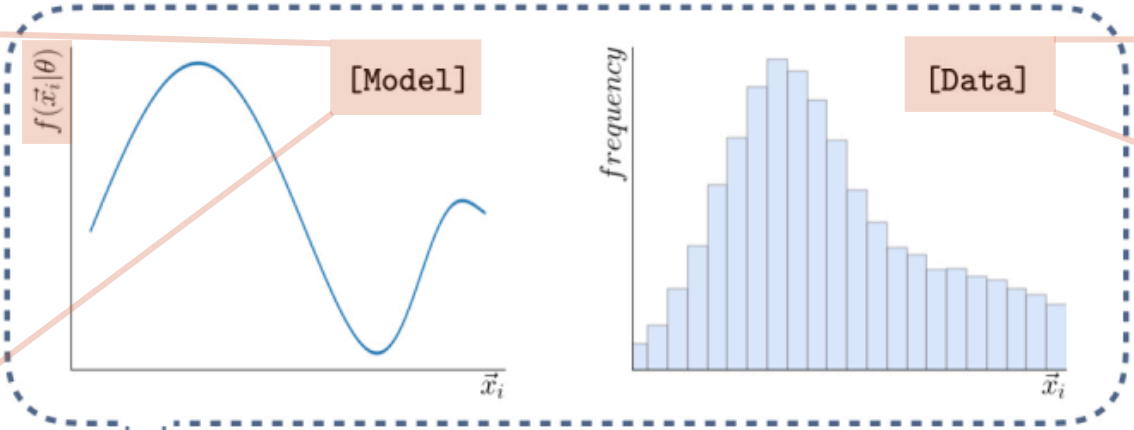
Pythonic
NumPy
python™
integrate into ecosystem, stable API

Deep Learning

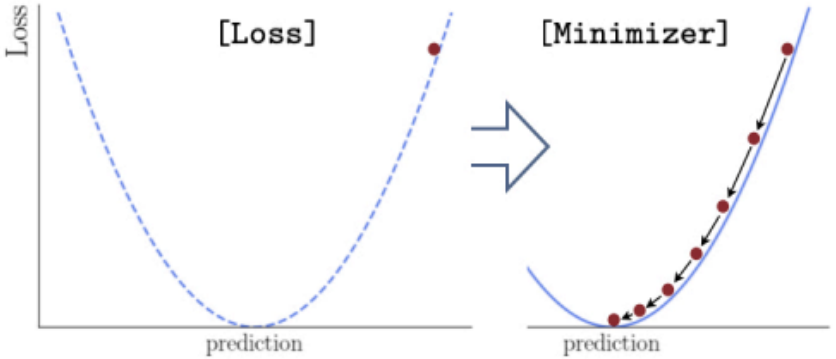
Neural Network



„One huge, complicated function“



„Big Data“



Deep Learning vs. Model Fitting

Similarity	Complicated Models	Large Data	Composed loss	Minimization	Results and uncertainties
HEP	Non-trivial functions	Whole Dataset	simultaneous, constraints	Global min, 2nd derivative algorithm	Hesse, profiling
Deep Learning	Combine many, trivial functions	Many, small Batches	*Anything! (GANs, RL,...)*	Local (!) min, 1th derivative, many steps	None
Conclusion					

Main backend: TensorFlow

- By Google, highly popular (150k★, top on)
- Consists of "two parts":
  - High level API for building neural networks *(NOT used!)*
  - Low level API with Numpy-style syntax

    tf.sqrt, tf.random.uniform,… or *tnp.sqrt, tnp.array, tnp.linspace*
- Two modes:
  - "numpy"-like (full Python flexibility)
  - "compiled" (very performant)

} GPU/Multi CPU support

JIT compilation

1) Function is called with arguments (array-like)
2) Function is ***traced with symbolic arrays***
   - Python code is executed
   - Only TF functions remembered
   - Cache compiled code with signature
3) Execute compiled code with inputs
4) Repeated calls: only step 1) and 3)

```python
@tf.function(autograph=False)
def add_mult(a, b, c, d):
    print("compiling...")
    tf.print("running")
    sum_ab = a + b
    sum_cd = c + d
    return sum_ab * sum_cd
```

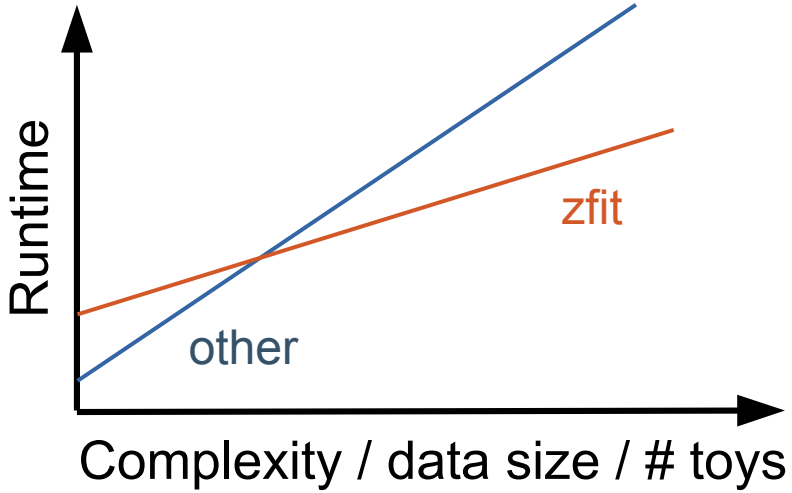
slow
```
add_mult(...)
# "compiling..."
# "running"
```

fast
```
add_mult(...)
# "running"
```

Scalable: Performance

Use same backend as ML uses

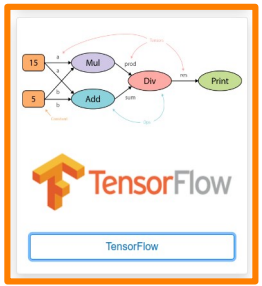
- Numpy-like backend TF (JAX, …)
  - JIT compiled
  - CPU and GPU
  - Automatic gradient
- Initial overhead, flat increase



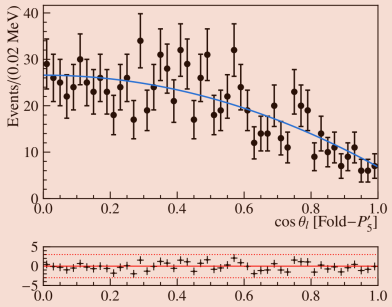
Single, simple fit "slow"

- 0.01 or 1 sec not relevant
- 1 or 10 hours relevant

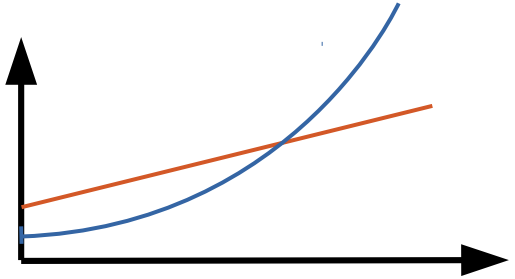
```
import zfit.z.numpy as znp


ar = znp.linspace(0, 1, 10)
sin = znp.sin(ar)
sum_sin = znp.sum(sin)
```



HEP Model Fitting in Python



HEP
advanced features,
simply extendable



Scalable
large data, complex models

Scikit HEP

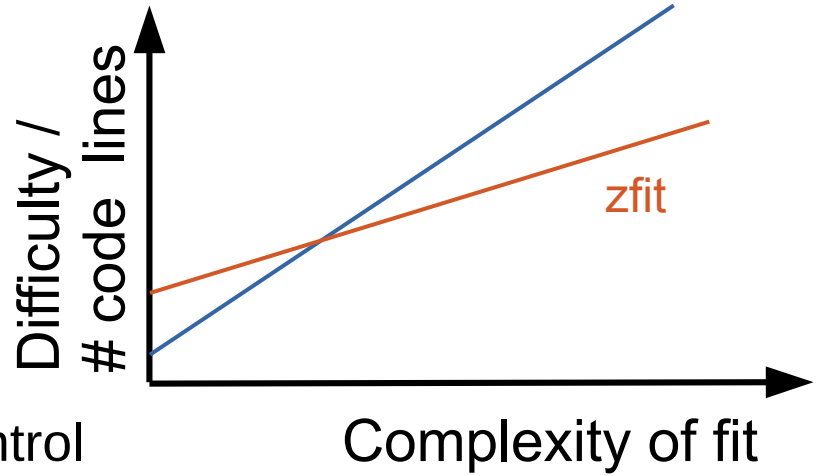
Pythonic
NumPy
python™
integrate into ecosystem, stable API

Scalable: Usability

«Things should not be easy or hard, but consistent»

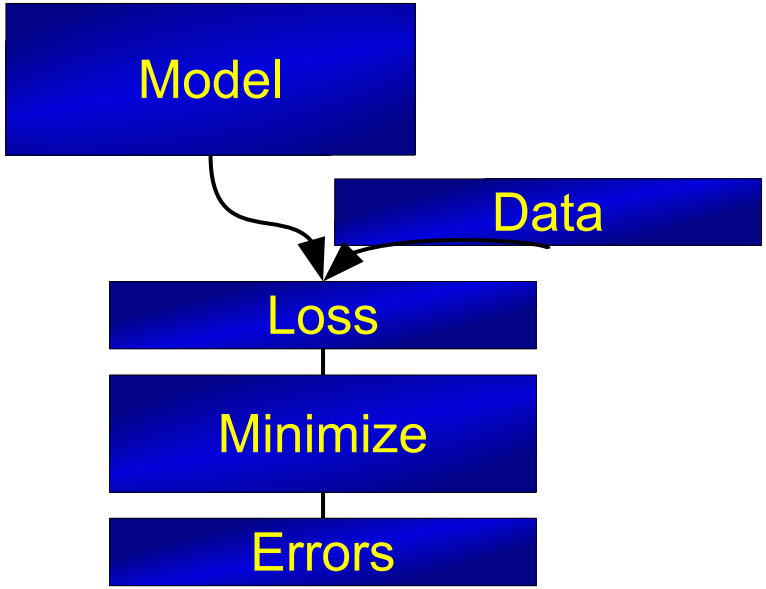
Cover all use-cases
out-of-the-box impossible

- Convenient base classes, allow full control
- **Modular structure**; provide all elements



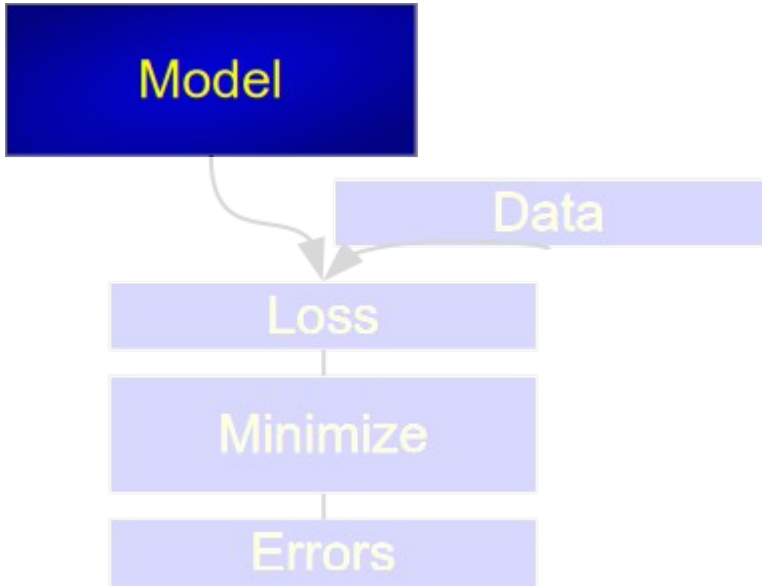
Basic API example

```
obs = zfit.Space("x", -2, 3)

mu = zfit.Parameter("mu", 1.2, -4, 6)
sigma = zfit.Parameter("sigma", 1.3, 0.5, 10)
gauss = zfit.pdf.Gauss(mu=mu, sigma=sigma, obs=obs)

data = zfit.Data(data, obs=obs)

nll = zfit.loss.UnbinnedNLL(model=gauss, data=data)

minimizer = zfit.minimize.Minuit()
result = minimizer.minimize(nll)

param_errors = result.hesse()
param_errors_asymmetric, new_result = result.errors()
```



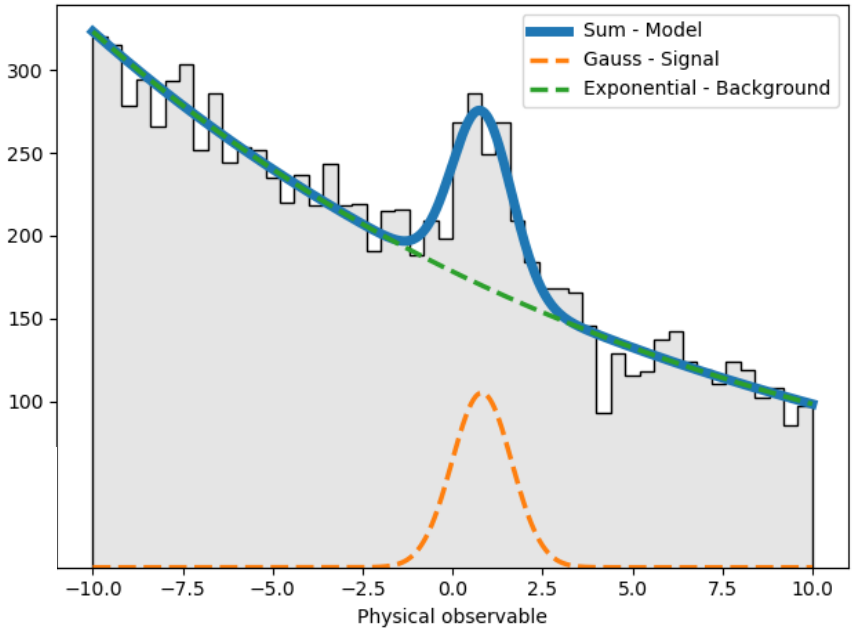
Basic API example

```
obs = zfit.Space("x", -2, 3)

mu = zfit.Parameter("mu", 1.2, -4, 6)
sigma = zfit.Parameter("sigma", 1.3, 0.5, 10)
gauss = zfit.pdf.Gauss(mu=mu, sigma=sigma, obs=obs)

data = zfit.Data(data, obs=obs)

nll = zfit.loss.UnbinnedNLL(model=gauss, data=data)

minimizer = zfit.minimize.Minuit()
result = minimizer.minimize(nll)

param_errors = result.hesse()
param_errors_asymmetric, new_result = result.errors()
```



Example fit

- Sum, Product, *(Convolution)*
- Gauss, (double) Crystalball,...
- Exponential, Polynomials,…
- Histograms, SplineInterpolation,...




```
lambd = zfit.Parameter("lambda", -0.06, -1, -0.01)
frac = zfit.Parameter("fraction", 0.3, 0, 1)

gauss = zfit.pdf.Gauss(mu=mu, sigma=sigma, obs=obs)
exponential = zfit.pdf.Exponential(lambd, obs=obs)
model = zfit.pdf.SumPDF([gauss, exponential], fracs=frac)
```

Custom PDF

```python
import zfit
import zfit.z.numpy as znp

class CustomPDF(zfit.pdf.ZPDF):
    _PARAMS = ("alpha",)

    @zfit.supports(norm=False)
    def _pdf(self, x, norm, params):
        data = x[0]  # axis 0
        alpha = params["alpha"]
        return znp.exp(alpha * data)
```

Example of zfit Base Classes

Can also override:

- integrate → _integrate
- pdf → _pdf
- sample → _sample

Or register integral

```python
custom_pdf = CustomPDF(obs=obs, alpha=0.2)

integral = custom_pdf.integrate(limits=(-1, 2))
sample   = custom_pdf.sample(n=1000)
prob     = custom_pdf.pdf(sample)
```

} use functionality of model

Arbitrary analytic shapes

PDFs TF-PWA RooFit pyhf ComPWA

Welcome to zfit-physics

zfit-physics is a package that provides physics-related functionality to zfit. It is not a standalone package but requires zfit to be installed.

It can be installed via pip:

```
pip install zfit-physics
```

or via conda:

```
conda install -c conda-forge zfit-physics
```

PDF documentation

PDFs

- `Argus`
- `CMSShape`
- `Cruijff`
- `ErfExp`
- `Novosibirsk`
- `RelativisticBreitWigner`
- `Tsallis`

For example, create amplitude with ComPWA and zfit

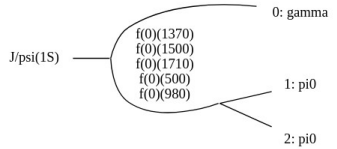
Amplitude analysis with zfit

▶ Show code cell content

Formulating the model

```
import qrules

reaction = qrules.generate_transitions(
    initial_state=("J/psi(1S)", [-1, +1]),
    final_state=["gamma", "pi0", "pi0"],
    allowed_intermediate_particles=["f(0)"],
    allowed_interaction_types=["strong", "EM"],
    formalism="helicity",
)
```

▶ Show code cell source




```
import ampform
from ampform.dynamics.builder import (
    create_non_dynamic_with_ff,
    create_relativistic_breit_wigner_with_ff,
)
```

Binned models

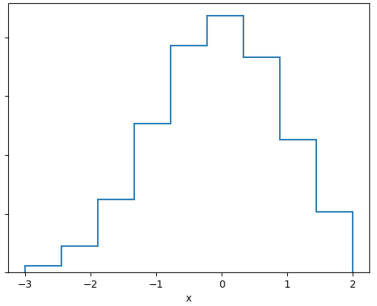
- Universal Histogram Interface (UHI)
- Modelled after **boost-histogram/hist/UHI**
  - Axes, names, ….


```
h = hist.Hist(hist.axis.Regular(3, -3, 3, name="x", flow=False),
              hist.axis.Regular(2, -5, 5, name="y", flow=False))
x = np.random.randn(1_000_000)
y = 0.5 * np.random.randn(1_000_000)
h.fill(x=x, y=y)

pdf = zfit.pdf.HistogramPDF(data=h)
```

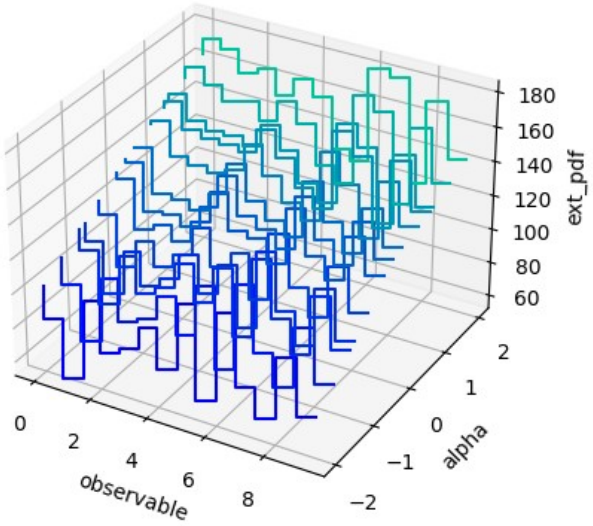
...and back

```
h_back = pdf.to_hist()
```

```
mplhep.histplot(h_back)
```

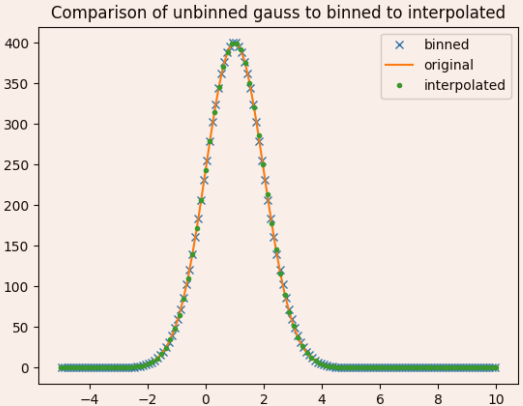


More histograms

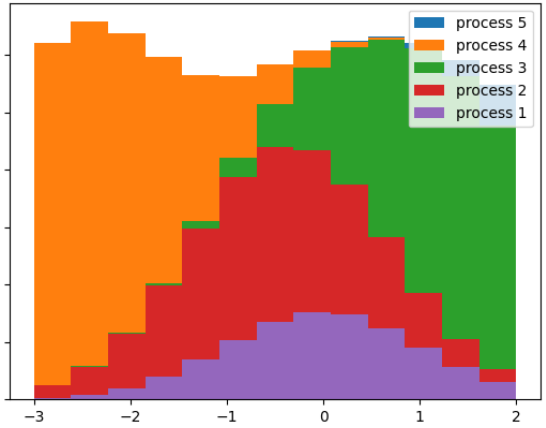


Shape modifier

```
pdf_syst = zfit.pdf.BinwiseScaleModifier(pdf, modifiers=True)
```



Unbinned → binned → interpolated

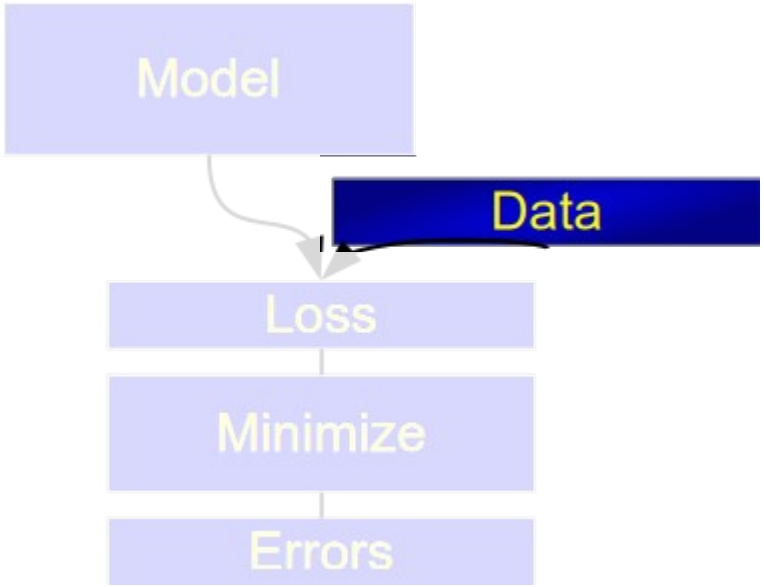



```
pdfs = [zfit.pdf.HistogramPDF(h) for h in histos]
alpha = zfit.Parameter('alpha', 0, -5, 5)
morph = SplineMorphingPDF(alpha=alpha, hists=pdfs)
```

```
pdfs = [zfit.pdf.HistogramPDF(h) for h in histos]
sumpdf = zfit.pdf.BinnedSumPDF(pdfs)
```

Basic API example

```
obs = zfit.Space("x", -2, 3)

mu = zfit.Parameter("mu", 1.2, -4, 6)
sigma = zfit.Parameter("sigma", 1.3, 0.5, 10)
gauss = zfit.pdf.Gauss(mu=mu, sigma=sigma, obs=obs)

data = zfit.Data(data, obs=obs)

nll = zfit.loss.UnbinnedNLL(model=gauss, data=data)

minimizer = zfit.minimize.Minuit()
result = minimizer.minimize(nll)

param_errors = result.hesse()
param_errors_asymmetric, new_result = result.errors()
```



Complete fit: Data

- From different sources
  - Hist, numpy, Pandas, ROOT, …

Python ecosystem for preprocessing

- Efficiently sampled from a model

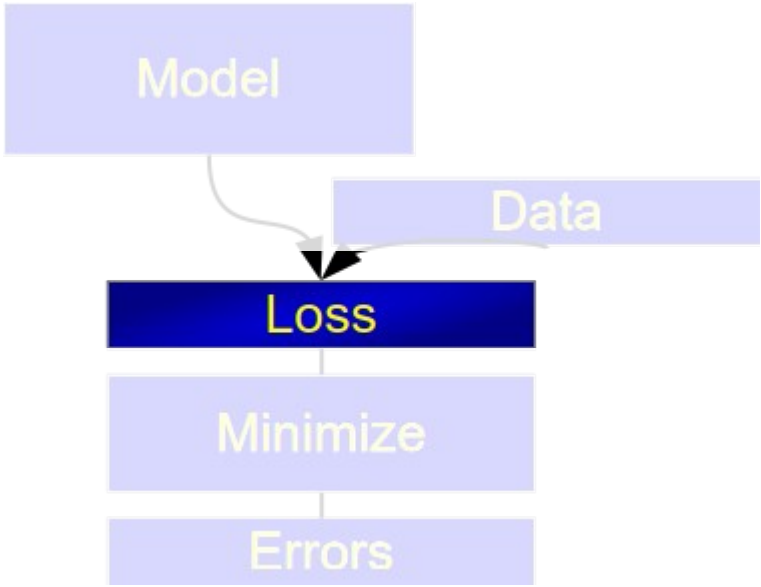
```
data = model.create_sampler(n_sample, limits=obs)
```

- UHI compatible!

```
binneddata = binnedpdf.sample()
mplhep.histplot(binneddata)
```

Basic API example

```
obs = zfit.Space("x", -2, 3)

mu = zfit.Parameter("mu", 1.2, -4, 6)
sigma = zfit.Parameter("sigma", 1.3, 0.5, 10)
gauss = zfit.pdf.Gauss(mu=mu, sigma=sigma, obs=obs)

data = zfit.Data(data, obs=obs)

nll = zfit.loss.UnbinnedNLL(model=gauss, data=data)

minimizer = zfit.minimize.Minuit()
result = minimizer.minimize(nll)

param_errors = result.hesse()
param_errors_asymmetric, new_result = result.errors()
```



Loss

```
nll1 = zfit.loss.UnbinnedNLL(model=gauss1, data=data1)
nll2 = zfit.loss.UnbinnedNLL(model=gauss2, data=data2)
nll_simultaneous2 = nll1 + nll2
```

(arbitrary) constraints supported, added to loss

```
constr = GaussianConstraint(params=params, observation=observed, uncertainty=sigma)
nll = zfit.loss.BinnedNLL(model=model, data=data, constraint=constr)
```

Available Loss Classes:

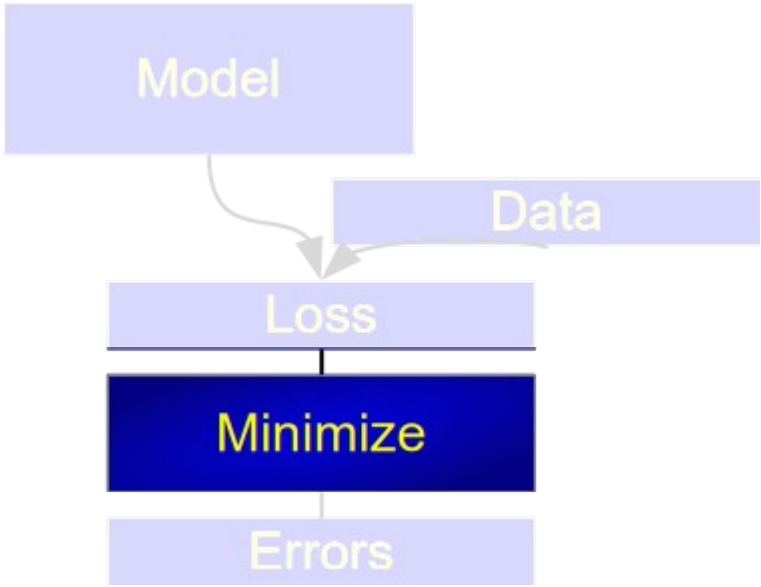
- `zfit.loss.UnbinnedNLL`
- `zfit.loss.ExtendedUnbinnedNLL`
- `zfit.loss.BinnedNLL`
- `zfit.loss.ExtendedBinnedNLL`
- `zfit.loss.BinnedChi2`

- `zfit.loss.ExtendedBinnedNLL`
- `zfit.loss.BinnedChi2`
- `zfit.loss.ExtendedBinnedChi2`
- `zfit.loss.BaseLoss`
- `zfit.loss.SimpleLoss`

Support sample weights

Basic API example

```
obs = zfit.Space("x", -2, 3)

mu = zfit.Parameter("mu", 1.2, -4, 6)
sigma = zfit.Parameter("sigma", 1.3, 0.5, 10)
gauss = zfit.pdf.Gauss(mu=mu, sigma=sigma, obs=obs)

data = zfit.Data(data, obs=obs)

nll = zfit.loss.UnbinnedNLL(model=gauss, data=data)

minimizer = zfit.minimize.Minuit()
result = minimizer.minimize(nll)

param_errors = result.hesse()
param_errors_asymmetric, new_result = result.errors()
```

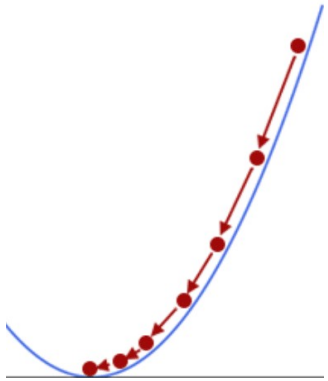


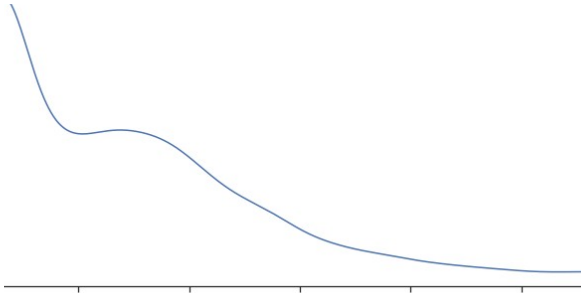
Minimize

A few, non-unified minimizer APIs in ecosystem

- Inferfaces non-optimal: many invalid combinations
- Different stopping criteria: xtol, ftol, absgtol, relxtol…

1. Configuration: differs partially
   - All have stopping criteria
2. Minimize: same





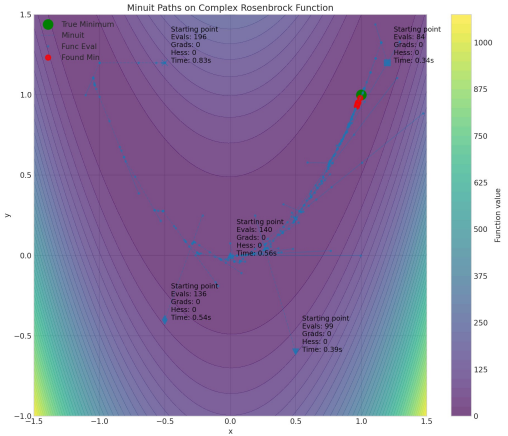
Minimize

config: differs partially

```
minimizer = zfit.minimize.IpyoptV1()
minimizer = zfit.minimize.Minuit()
minimizer = zfit.minimize.ScipyTrustConstrV1()
minimizer = zfit.minimize.NLoptLBFGSV1()
```

minimize(...): same API

```
result = minimizer.minimize(func, params)
```



class **zfit.minimize.EDM**(*tol*, *loss*, *params*, *name='edm'*)

Bases: `ConvergenceCriterion`

Estimated distance to minimum.

This criterion estimates the distance to the minimum by using

$$EDM = g^T \cdot H^{-1} \cdot g$$

with H the hessian matrix (approximation) and g the gradient.

ScipyLBFGSB

class **zfit.minimize.ScipyLBFGSB**(*tol=None*, *maxcor=None*, *maxls=None*, *verbosity=None*, *gradient=None*, *maxiter=None*, *criterion=None*, *strategy=None*, *name='SciPy L-BFGS-B '*) [source]

Bases: `ScipyBaseMinimizer`

Local, gradient based quasi-Newton algorithm using the limited-memory BFGS approximation.

Limited-memory BFGS is an optimization algorithm in the family of quasi-Newton methods that approximates the Broyden-Fletcher-Goldfarb-Shanno algorithm (BFGS) using a limited amount of memory (or gradients, controlled by *maxcor*).

L-BFGS borrows ideas from the trust region methods while keeping the L-BFGS update of the Hessian and line search algorithms.

This implenemtation wraps the minimizers in SciPy optimize.

Parameters:

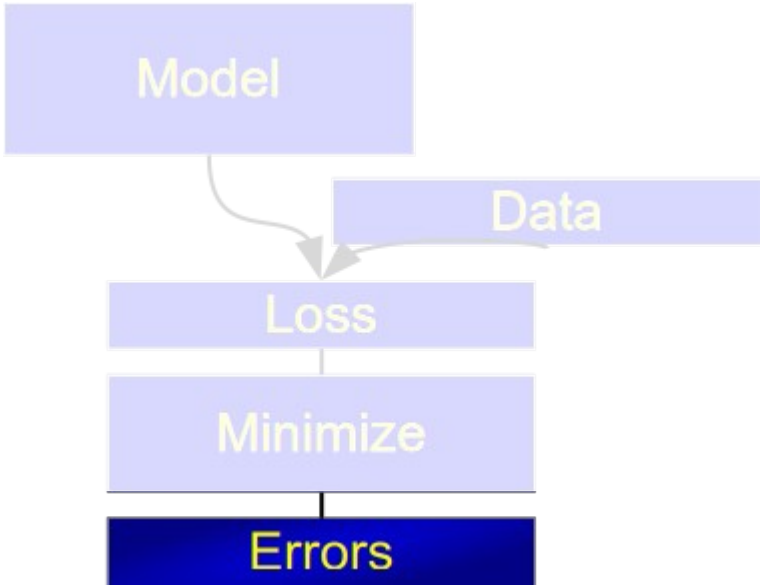
- **tol** (`float` | `None`) – Termination value for the convergence/stopping criterion of the algorithm in order to determine if the minimum has been found. Defaults to 1e-3.
- **maxcor** (`int` | `None`) – Maximum number of memory history to keep when using a quasi-Newton update formula such as BFGS. It is the number of gradients to "remember" from previous optimization steps: increasing it increases the memory requirements but may speed up the convergence.
- **maxls** (`int` | `None`) – Maximum number of linesearch points.
- **verbosity** (`int` | `None`) –

  Verbosity of the minimizer. Has to be between 0 and 10. The verbosity has the meaning:

Can use zfit loss, but also ***pure Python function***

Basic API example

```
obs = zfit.Space("x", -2, 3)

mu = zfit.Parameter("mu", 1.2, -4, 6)
sigma = zfit.Parameter("sigma", 1.3, 0.5, 10)
gauss = zfit.pdf.Gauss(mu=mu, sigma=sigma, obs=obs)

data = zfit.Data(data, obs=obs)

nll = zfit.loss.UnbinnedNLL(model=gauss, data=data)

minimizer = zfit.minimize.Minuit()
result = minimizer.minimize(nll)

param_errors = result.hesse()
param_errors_asymmetric, new_result = result.errors()
```



Serialization

1) Dump/load with dill

```
result_dilled = zfit.dill.dumps(result)
result_loaded = zfit.dill.loads(result_dilled)
```

2) Dump/load human readable

```
model_yaml = model.to_json()
model_loaded = zfit.pdf.SumPDF.from_json(model_yaml)

hs3like = zfit.hs3.dumps(nll)
nll_loaded = zfit.hs3.loads(hs3like)
```

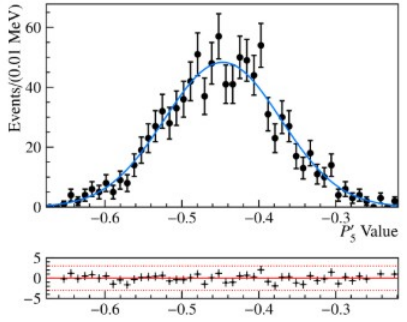

```
'pdfs': {'SumPDF': {'pdfs': [{'extended': 'n_sig',
                              'mu': 'mu',
                              'sigma': 'sigma',
                              'type': 'Gauss',
                              'x': 'x'},
                             {'extended': 'n_bkg',
                              'lam': 'lambda',
                              'type': 'Exponential',
                              'x': 'x'}],
                    'type': 'SumPDF'}},
'variables': {'lambda': {'max': -0.009999999776482582,
                         'min': -1.0,
                         'name': 'lambda',
                         'step_size': 0.001,
                         'value': -0.06294756382703781},
```

zfit-tutorials

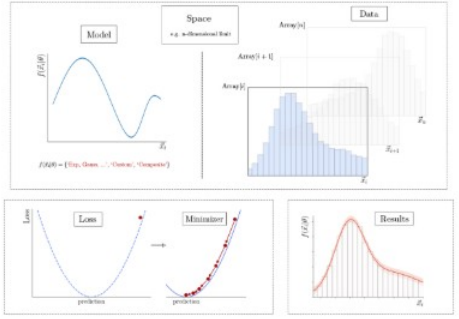
Search

Ctrl + K

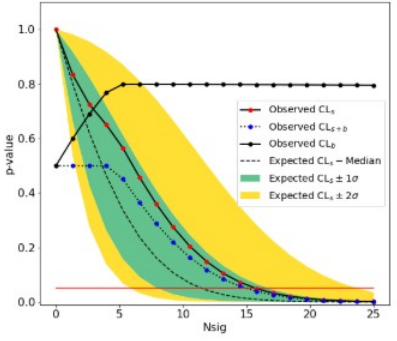
- Introduction
- Components
- Guides
- HPC with TensorFlow



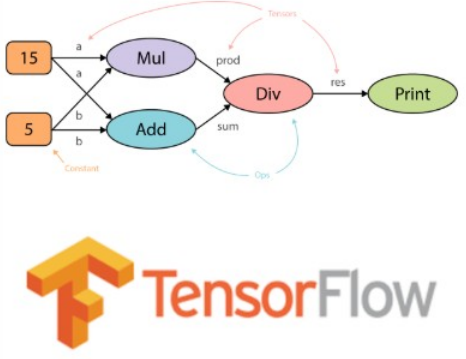
Introduction



Components



Guides



TensorFlow

TensorFlow

Conclusion

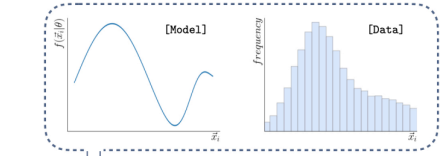
General likelihood fitter for complicated models with numerical support

Well integrated with the Python ecosystem

JIT compiled, autograd, GPU support

Approaching 1.0 release

Future plans: JAX based rewrite of major components



	C++ library	Numpy based	zfit
HEP specific content/API			
Models		SciPy	TF Probability
Gradients	CLAD		
Computational optimizations		Numba NumPy	TensorFlow NVIDIA
Parallelization/GPU			
Low level handling		python	